



SECURE AUTOMATED WEB ARCHITECTURE

*A DevSecOps Capstone: Infrastructure as Code, Automated Security Scanning,
and a Live, Production-Style AWS Deployment*

TKH Innovation Fellowship | Phase 2: Cloud Security Engineering | Sprint 6 Capstone

Presented by: Team E

01–02

Mission Briefing

Chad Bachoo

03–04

Milestone 1: Infrastructure

Alexandra Blandon

05–06

Milestone 2: The Pipeline

Isabelli Duran

07–08

Milestone 3: Deployment

Bineta Fall

09–10

Teardown & Takeaways

Alexis Weathers

Build It Like a Hiring Manager Is Watching



The Scenario

As Lead Architect for a startup, our job was to go from a blank canvas to a fully automated, secure cloud web stack — exactly as if we were shipping this for a real employer, not just submitting a school assignment.

Three milestones, three nights: write and validate the Terraform, wire a security scanner into the pipeline so bad code can never ship, then deploy it live and document it like a portfolio piece.

What “Exceeds Expectations” Looks Like



Zero manual console clicks — everything provisioned through code



A pipeline that physically breaks the build on real vulnerabilities

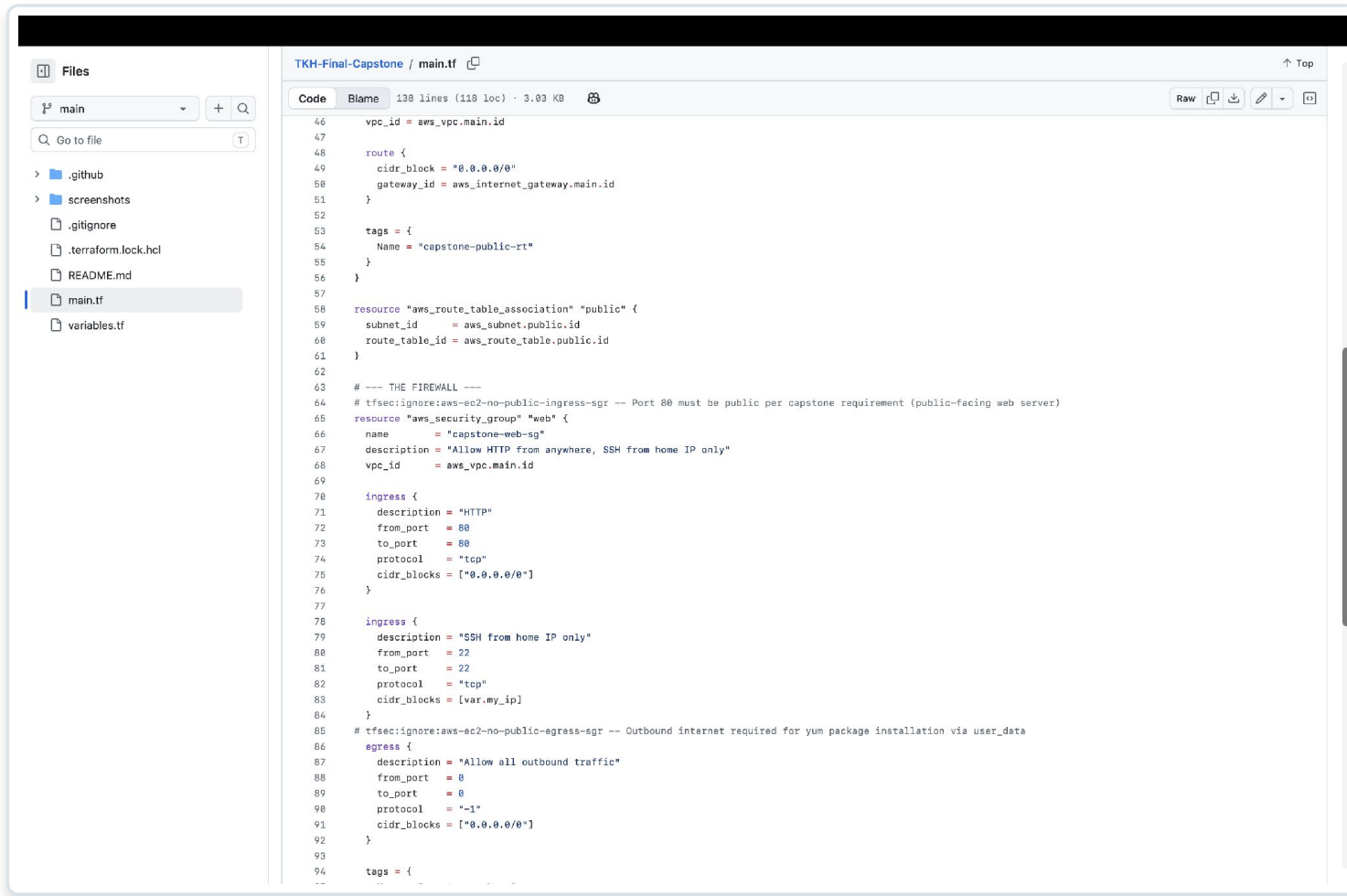


Least-privilege network access, not just a working server



Documentation clear enough for someone outside the classroom

The Network Blueprint



```
46 vpc_id = aws_vpc.main.id
47
48 route {
49   cidr_block = "0.0.0.0/0"
50   gateway_id = aws_internet_gateway.main.id
51 }
52
53 tags = {
54   Name = "capstone-public-rt"
55 }
56 }
57
58 resource "aws_route_table_association" "public" {
59   subnet_id      = aws_subnet.public.id
60   route_table_id = aws_route_table.public.id
61 }
62
63 # --- THE FIREWALL ---
64 # tfsec:ignore:aws-ec2-no-public-ingress-sgr -- Port 80 must be public per capstone requirement (public-facing web server)
65 resource "aws_security_group" "web" {
66   name        = "capstone-web-sg"
67   description = "Allow HTTP from anywhere, SSH from home IP only"
68   vpc_id      = aws_vpc.main.id
69
70   ingress {
71     description = "HTTP"
72     from_port   = 80
73     to_port     = 80
74     protocol    = "tcp"
75     cidr_blocks = ["0.0.0.0/0"]
76   }
77
78   ingress {
79     description = "SSH from home IP only"
80     from_port   = 22
81     to_port     = 22
82     protocol    = "tcp"
83     cidr_blocks = [var.my_ip]
84   }
85   # tfsec:ignore:aws-ec2-no-public-egress-sgr -- Outbound internet required for yum package installation via user_data
86   egress {
87     description = "Allow all outbound traffic"
88     from_port   = 0
89     to_port     = 0
90     protocol    = "-1"
91     cidr_blocks = ["0.0.0.0/0"]
92   }
93
94   tags = {
```



Custom VPC, not the default

10.0.0.0/16 with a dedicated 10.0.1.0/24 public subnet — nothing shared, nothing assumed.

Explicit routing

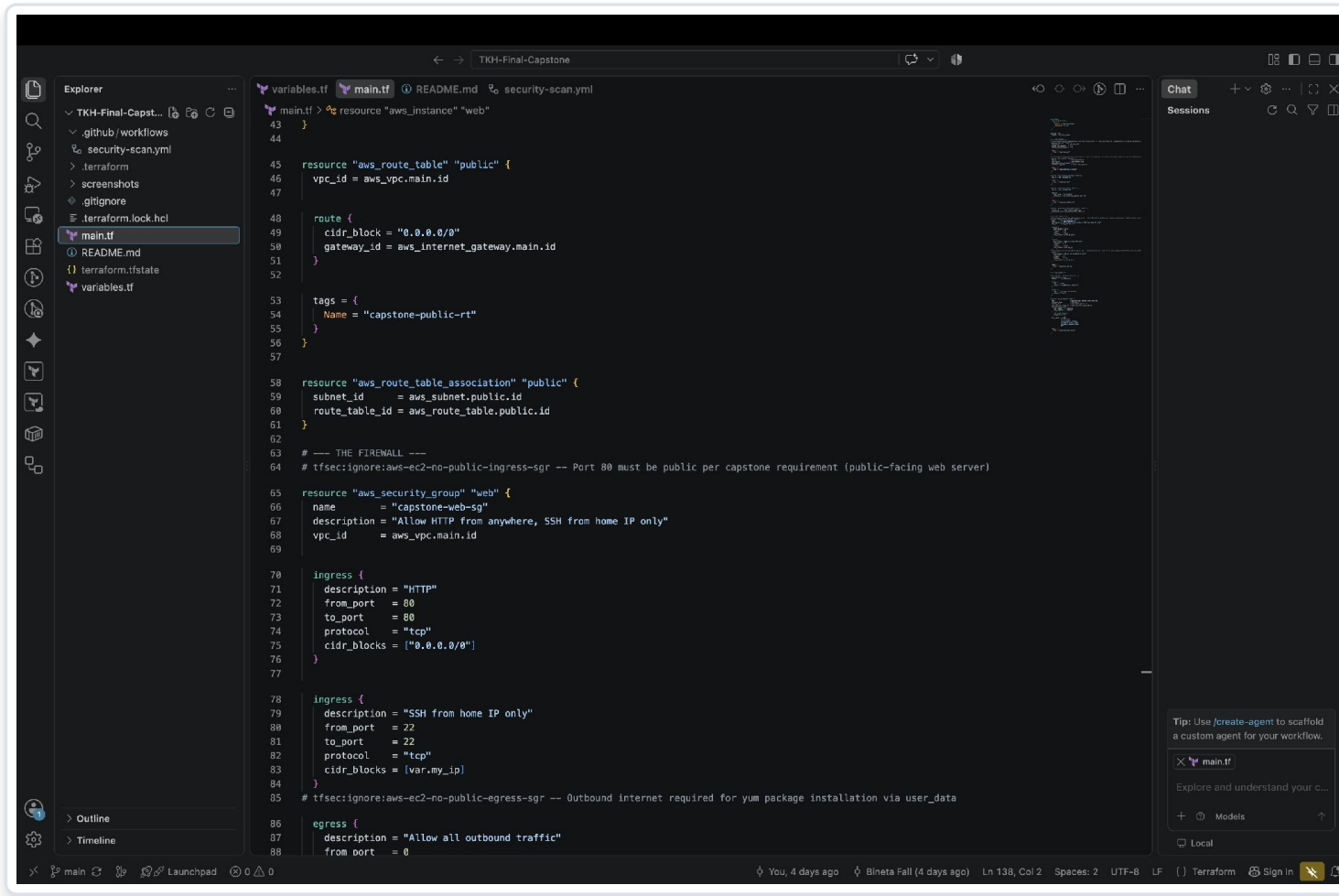
A route table sends 0.0.0.0/0 through our Internet Gateway, associated directly to the public subnet.

Every risk, commented

Where a requirement forces an open rule, we document why right in the code — not just why it works.

main.tf — route table, route table association, and the start of the security group definition

Locking Down the Firewall



```
43 }
44
45 resource "aws_route_table" "public" {
46   vpc_id = aws_vpc.main.id
47
48   route {
49     cidr_block = "0.0.0.0/0"
50     gateway_id = aws_internet_gateway.main.id
51   }
52
53   tags = {
54     Name = "capstone-public-rt"
55   }
56 }
57
58 resource "aws_route_table_association" "public" {
59   subnet_id = aws_subnet.public.id
60   route_table_id = aws_route_table.public.id
61 }
62
63 # --- THE FIREWALL ---
64 # tfsec:ignore:aws-ec2-no-public-ingress-sgr -- Port 80 must be public per capstone requirement (public-facing web server)
65
66 resource "aws_security_group" "web" {
67   name = "capstone-web-sg"
68   description = "Allow HTTP from anywhere, SSH from home IP only"
69   vpc_id = aws_vpc.main.id
70
71   ingress {
72     description = "HTTP"
73     from_port = 80
74     to_port = 80
75     protocol = "tcp"
76     cidr_blocks = ["0.0.0.0/0"]
77   }
78
79   ingress {
80     description = "SSH from home IP only"
81     from_port = 22
82     to_port = 22
83     protocol = "tcp"
84     cidr_blocks = [var.my_ip]
85   }
86
87   # tfsec:ignore:aws-ec2-no-public-egress-sgr -- Outbound internet required for yum package installation via user_data
88   egress {
89     description = "Allow all outbound traffic"
90     from_port = 0
91     to_port = 0
92     protocol = "all"
93     cidr_blocks = ["0.0.0.0/0"]
94   }
95 }
```



Port 80: open, on purpose

The web server has to be public — that's the whole point — so HTTP ingress is intentionally 0.0.0.0/0.

Port 22: locked to one IP

SSH is restricted to a single trusted home IP address instead of the entire internet.

Least privilege, documented

Every accepted exception is a code comment tied to a real tfsec rule ID, not a silent override.

main.tf — aws_security_group "web", with justified tfsec:ignore annotations

The Quality Gate Fails — By Design

The screenshot shows a GitHub Actions workflow run for 'DevSecOps Quality Gate'. The workflow has a status of 'Failed' with a warning icon. The left sidebar shows the workflow details, including a summary, all jobs, and a list of steps. The main panel shows the 'Run tfsec' step, which failed. The failure message is '1 warning'. The logs show the setup of the workflow, including the installation of tfsec. The logs are as follows:

```
1 Run aquasecurity/tfsec-pr-commenter-action@v1.2.0
2 /usr/bin/docker run --name d41bcc385a1a26014a3689389dc0f0e61ec_f5d3f4 --label 8d41bc --workdir /github/workspace --rm -e "INPUT_TFSEC_ARGS" -e
"INPUT_GITHUB_TOKEN" -e "INPUT_WORKING_DIRECTORY" -e "INPUT_TFSEC_VERSION" -e "INPUT_TFSEC_FORMATS" -e "INPUT_SOFT_FAIL_COMMENTER" -e
"HOME" -e "GITHUB_JOB" -e "GITHUB_REF" -e "GITHUB_SHA" -e "GITHUB_REPOSITORY" -e "GITHUB_REPOSITORY_OWNER" -e "GITHUB_REPOSITORY_OWNER_ID" -e "GITHUB_RUN_ID" -e
"GITHUB_RUN_NUMBER" -e "GITHUB_RETENTION_DAYS" -e "GITHUB_RUN_ATTEMPT" -e "GITHUB_ACTOR_ID" -e "GITHUB_ACTOR" -e "GITHUB_WORKFLOW" -e "GITHUB_HEAD_REF" -e
"GITHUB_BASE_REF" -e "GITHUB_EVENT_NAME" -e "GITHUB_SERVER_URL" -e "GITHUB_API_URL" -e "GITHUB_GRAPHQL_URL" -e "GITHUB_REF_NAME" -e "GITHUB_REF_PROTECTED" -e
"GITHUB_REF_TYPE" -e "GITHUB_WORKFLOW_REF" -e "GITHUB_WORKFLOW_SHA" -e "GITHUB_REPOSITORY_ID" -e "GITHUB_TRIGGERING_ACTOR" -e "GITHUB_WORKSPACE" -e "GITHUB_ACTION"
-e "GITHUB_EVENT_PATH" -e "GITHUB_ACTION_REPOSITORY" -e "GITHUB_ACTION_REF" -e "GITHUB_PATH" -e "GITHUB_ENV" -e "GITHUB_STEP_SUMMARY" -e "GITHUB_STATE" -e
"GITHUB_OUTPUT" -e "GITHUB_ARTIFACTS" -e "GITHUB_ARTIFACTS_LIST" -e "RUNNER_OS" -e "RUNNER_ARCH" -e "RUNNER_NAME" -e "RUNNER_ENVIRONMENT" -e "RUNNER_TOOL_CACHE" -e
"RUNNER_TEMP" -e "RUNNER_WORKSPACE" -e "ACTIONS_RUNTIME_URL" -e "ACTIONS_RUNTIME_TOKEN" -e "ACTIONS_CACHE_URL" -e "ACTIONS_RESULTS_URL" -e
"ACTIONS_ORCHESTRATION_ID" -e GITHUB_ACTIONS=true -e CI=true -v "/var/run/docker.sock":"/var/run/docker.sock" -v "/home/runner/work/_temp":"/github/runner_temp" -v
"/home/runner/work/_temp/github_home":"/github/home" -v "/home/runner/work/_temp/github_workflow":"/github/workflow" -v
"/home/runner/work/_temp/runner_file_commands":"/github/file_commands" -v "/home/runner/work/TKH-Final-Capstone/TKH-Final-Capstone":"/github/workspace"
8d41bc:c385a1a26014a3689389dc0f0e61ec
9 + TFSEC_VERSION=
10 + '[' latest != latest ']'
11 ++ wget -q https://api.github.com/repos/aquasecurity/tfsec/releases -O -
12 ++ grep -m 1 -o -E 'https://.+?tfsec-linux-amd64'
13 ++ head -n1
14 + wget -O - -q https://github.com/aquasecurity/tfsec/releases/download/v1.28.14/tfsec-linux-amd64
15 ++ wget -q https://api.github.com/repos/aquasecurity/tfsec/releases -O -
16 ++ grep -m 1 -o -E 'https://.+?tfsec_checksums.txt'
17 ++ head -n1
18 + wget -O - -q https://github.com/aquasecurity/tfsec/releases/download/v1.28.14/tfsec_checksums.txt
19 + grep tfsec-linux-amd64 tfsec.checksums
20 + sha256sum -c tfsec-linux-amd64.checksum
21 tfsec-linux-amd64: OK
22 + install tfsec-linux-amd64 /usr/local/bin/tfsec
23 . . .
```



Soft-fail is off

Our GitHub Actions workflow runs tfsec on every push to main with `--soft-fail=false`.

It broke on the first try

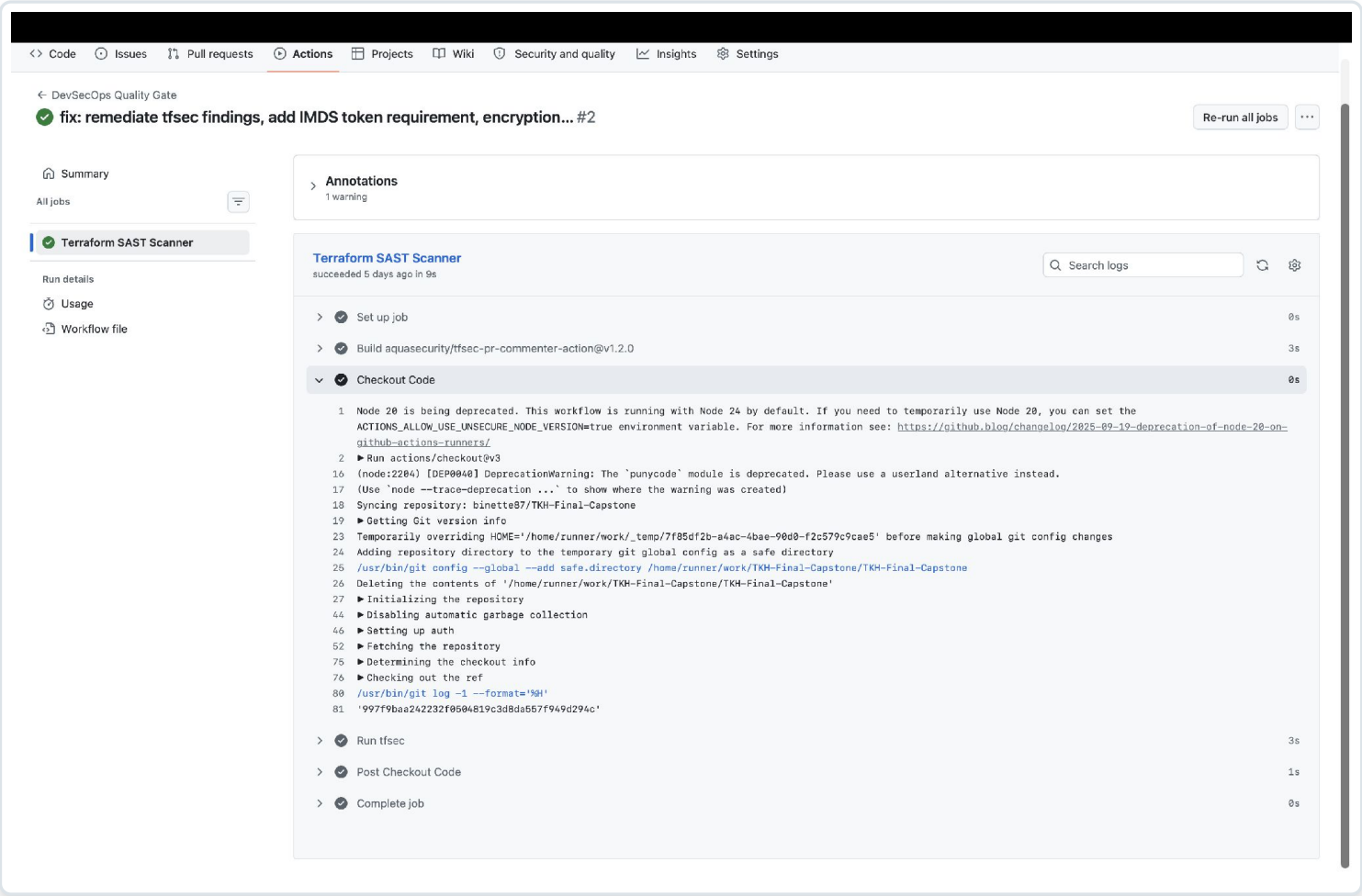
2 critical and 3 high-severity findings — an open security group rule and an unencrypted root volume.

That's the pipeline working

A quality gate that never fails isn't checking anything. This is exactly what “physically breaks the build” means.

GitHub Actions — first pipeline run, tfsec scanning main.tf on push

Remediation → Green Build



Real fixes, not suppressions

Enforced IMDSv2 tokens, enabled root volume encryption, and added rule descriptions.

Accepted risk, documented

The two findings required by the project scope were kept and justified with tfsec:ignore comments, not deleted.

0 critical · 0 high · 0 medium · 0 low

Same scanner, same code style — the difference is deliberate, informed security decisions.

GitHub Actions — Terraform SAST Scanner, succeeded in 9 seconds

Live Verification on AWS

It works!



One command to deploy

terraform apply -auto-approve provisioned the VPC, security group, and EC2 instance end to end.

Self-configuring server

A user_data script installs and starts Apache automatically the moment the instance boots.

Proof, not a promise

This page loading confirms both the network routing and the bootstrap script worked exactly as designed.

The live public IP, loaded in-browser — zero manual server setup

The Repository & README

The screenshot displays a GitHub repository named 'TKH-Final-Capstone' by user 'binette87'. The repository is public and has 5 commits. The commit history table shows the following files and their last commit dates:

File	Commit Message	Commit Date
.github	Added README and finalized project	4 days ago
screenshots	Add live deployment screenshot to README	4 days ago
.gitignore	edited: W12 S34 Terraform blueprint for VPC, security g...	5 days ago
.terraform.lock.hcl	edited: W12 S34 Terraform blueprint for VPC, security g...	5 days ago
README.md	Add live deployment screenshot to README	4 days ago
main.tf	fix: remediate tfsec findings, add IMDS token requirement,...	5 days ago
variables.tf	edited: W12 S34 Terraform blueprint for VPC, security g...	5 days ago

The README file is open, showing the title 'Secure Automated Web Architecture'. The description states: 'This project provisions a secure, publicly accessible web server on AWS using Terraform, with an automated CI/CD pipeline that scans every code change for security vulnerabilities before deployment. It demonstrates infrastructure as code, DevSecOps pipeline integration, and secure network design from a blank canvas to a live, production-style deployment.' The 'Live Deployment Proof' section includes a placeholder image.



Written for a hiring manager

Project description, tech stack, and architecture — understandable outside the classroom.

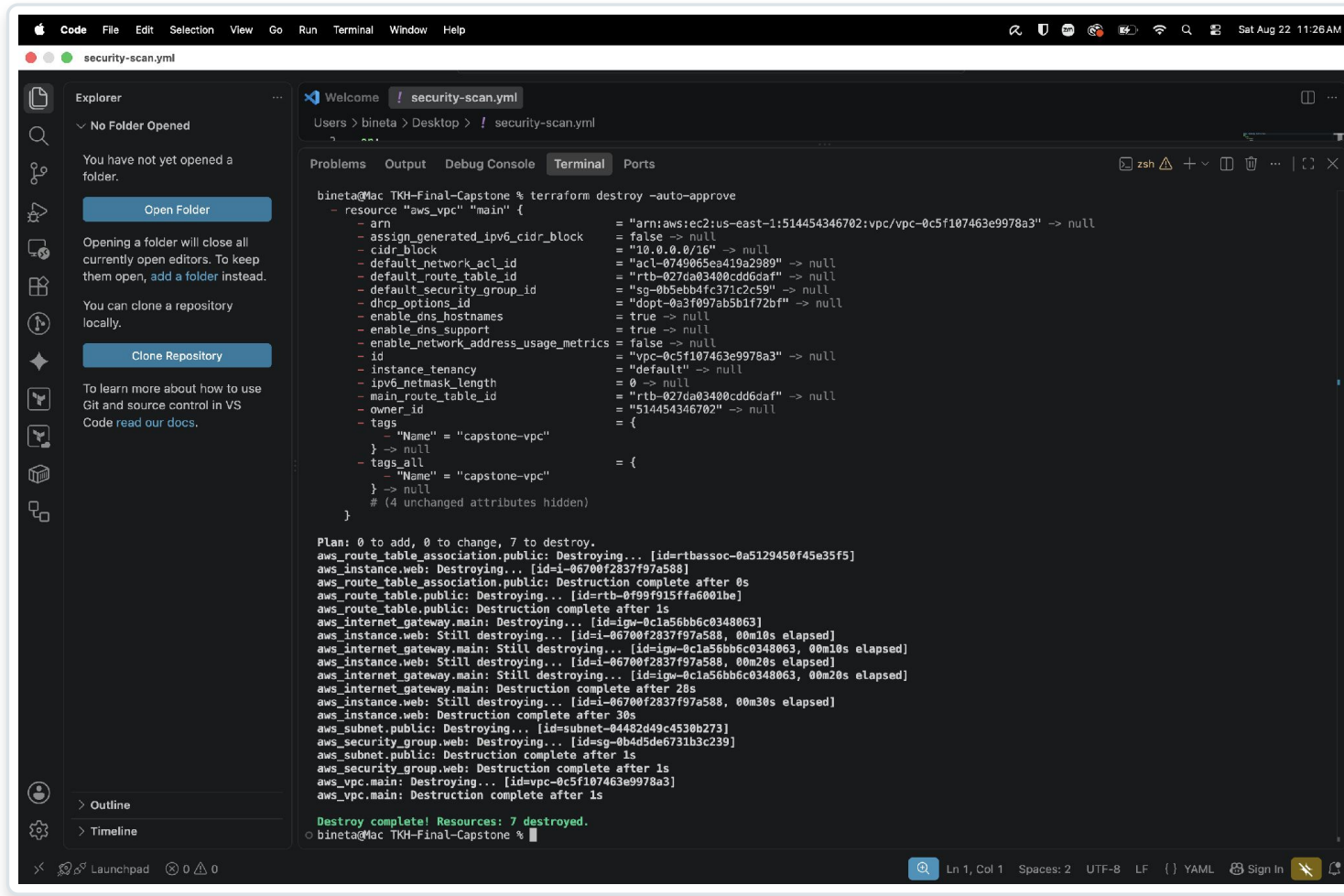
Security notes, not just a diagram

The README explicitly lists which tfsec findings were fixed and which were accepted, and why.

Proof embedded, not just claimed

The live deployment screenshot lives directly in the README, next to the explanation it supports.

Burning the Ships



```
bineta@Mac TKH-Final-Capstone % terraform destroy -auto-approve
- resource "aws_vpc" "main" {
  - arn                    = "arn:aws:ec2:us-east-1:514454346702:vpc/vpc-0c5f107463e9978a3" -> null
  - assign_generated_ip   = false -> null
  - cidr_block            = "10.0.0.0/16" -> null
  - default_network_acl_id = "acl-0749065ea419a2989" -> null
  - default_route_table_id = "rtb-027da03400cdd6daf" -> null
  - default_security_group_id = "sg-0b5ebb4fc371c2c59" -> null
  - dhcp_options_id      = "dopt-0a3f097ab5b1f72bf" -> null
  - enable_dns_hostnames = true -> null
  - enable_dns_support   = true -> null
  - enable_network_address_usage_metrics = false -> null
  - id                   = "vpc-0c5f107463e9978a3" -> null
  - instance_tenancy     = "default" -> null
  - ipv6_netmask_length  = 0 -> null
  - main_route_table_id  = "rtb-027da03400cdd6daf" -> null
  - owner_id             = "514454346702" -> null
  - tags                 = {
    - "Name" = "capstone-vpc"
  } -> null
  - tags_all = {
    - "Name" = "capstone-vpc"
  } -> null
  # (4 unchanged attributes hidden)
}

Plan: 0 to add, 0 to change, 7 to destroy.
aws_route_table_association.public: Destroying... [id=rtbassoc-0a5129450f45e35f5]
aws_instance.web: Destroying... [id=i-06700f2837f97a588]
aws_route_table_association.public: Destruction complete after 0s
aws_route_table.public: Destroying... [id=rtb-0f99f915ffa6001be]
aws_route_table.public: Destruction complete after 1s
aws_internet_gateway.main: Destroying... [id=igw-0cla56bb6c0348063]
aws_instance.web: Still destroying... [id=i-06700f2837f97a588, 00m10s elapsed]
aws_internet_gateway.main: Still destroying... [id=igw-0cla56bb6c0348063, 00m10s elapsed]
aws_instance.web: Still destroying... [id=i-06700f2837f97a588, 00m20s elapsed]
aws_internet_gateway.main: Still destroying... [id=igw-0cla56bb6c0348063, 00m20s elapsed]
aws_internet_gateway.main: Destruction complete after 28s
aws_instance.web: Still destroying... [id=i-06700f2837f97a588, 00m30s elapsed]
aws_instance.web: Destruction complete after 30s
aws_subnet.public: Destroying... [id=subnet-04482d49c4530b273]
aws_security_group.web: Destroying... [id=sg-0b4d5de6731b3c239]
aws_subnet.public: Destruction complete after 1s
aws_security_group.web: Destruction complete after 1s
aws_vpc.main: Destroying... [id=vpc-0c5f107463e9978a3]
aws_vpc.main: Destruction complete after 1s

Destroy complete! Resources: 7 destroyed.
```



Recorded first, destroyed second

The video walkthrough was saved and verified before a single resource was torn down.

One command, full cleanup

terraform destroy removed the VPC, subnet, security group, instance, and gateway — nothing left running.

Destroy complete: 7 resources

Confirmed in both the terminal output and the AWS console — billing stops immediately.

terraform destroy -auto-approve — all 7 resources removed after recording the demo



Key Takeaways



Security as code, not an afterthought

Every accepted risk lives in the codebase itself — reviewable, versioned, and justified.



A pipeline that earns trust by failing first

Our first real bugs were caught by the scanner, not by a hiring manager — or an attacker.



Built once, run anywhere

From blank canvas to live deployment, everything is reproducible with three Terraform commands.

Thank you — questions?